

Applications Development Practice 4 (ADP4) - Term 1 Notes

This study guide consolidates the Term 1 curriculum as outlined in the subject specifications. It focuses on the foundational algorithmic concepts, complexity analysis, searching mechanisms, and essential linear data structures needed for your upcoming exam.

1. Introduction to Algorithms and Complexity (Big O Notation)

Algorithm analysis provides a theoretical framework to predict the performance of a program as the size of the input data (N) increases. The two primary metrics evaluated are **Time Complexity** (execution time) and **Space Complexity** (memory consumption).

- **$O(1)$ - Constant Time:** Execution time remains the same regardless of input size.
- **$O(N)$ - Linear Time:** Execution time grows directly in proportion to the input size.
- **$O(\log N)$ - Logarithmic Time:** Execution time grows very slowly; typical of divide-and-conquer algorithms like Binary Search.
- **$O(N^2)$ - Quadratic Time:** Execution time grows exponentially, common in algorithms using nested loops (like Bubble Sort).

2. Recursion vs. Iteration

Recursion occurs when a function calls itself to solve a smaller instance of the same problem. Iteration, by contrast, relies on loop constructs (for, while) to repeat a block of code.

- **Factorial:** Mathematically defined as $n! = n * (n-1)!$. This is a classic recursive problem where the base case is defined as $0! = 1$.
- **Fibonacci Sequence:** A sequence where each number is the sum of the two preceding ones. While it can be elegantly written with recursion, a naive recursive approach has a severe time complexity footprint due to calculating the same subproblems repeatedly.

Key Comparison: Recursion often leads to cleaner, more readable code but uses significantly more memory due to Call Stack overhead, risking StackOverflow exceptions if the base case is missing or incorrect.

3. Searching Algorithms

Term 1 introduces the concept of algorithmic complexity through the comparison of two primary searching techniques:

Algorithm	Mechanism	Time Complexity	Prerequisites
Linear Search	Iterates sequentially through every element in a data structure until the target value is found.	$O(N)$	None (Works well on unsorted data).
Binary Search	Repeatedly divides the search interval in half. If the target is less than the middle element, it searches the lower half, discarding the upper half.	$O(\log N)$	Data MUST be sorted beforehand.

4. Sorting Algorithms: Bubble Sort

Bubble Sort is an introductory algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. With each pass, the largest unsorted element "bubbles" to its correct position at the end of the array.

- **Best-Case Complexity:** $O(N)$ — This only occurs when the array is already perfectly sorted (assuming the algorithm is optimized with a "swap flag" to break early).
- **Worst-Case Complexity:** $O(N^2)$ — Occurs when the array is sorted in reverse order.
- **Average-Case Complexity:** $O(N^2)$ — Due to the high volume of comparisons and swaps, it is highly inefficient for large datasets.

5. Data Structures: Arrays vs. Linked Lists

5.1 2D and Multidimensional Arrays

Arrays store elements in contiguous memory blocks. A 2D array typically represents data in a

grid (matrix) structure consisting of rows and columns. **Limitations:** Arrays have a fixed size declared at compile time. Furthermore, inserting or deleting an element in the middle of an array is computationally expensive because subsequent elements must be shifted.

5.2 Linear Linked Lists

Linked Lists resolve the size limitation of arrays by using dynamic memory allocation. Each element (a Node) stores the data payload alongside a pointer (reference) to the memory address of the next node. The memory does not need to be contiguous. **Limitation of a Single Linked List:** Traversal is strictly unidirectional. If you pass a node, you cannot step backward; you must restart traversal from the Head node.

5.3 Doubly and Circular Linked Lists

These structures are introduced to overcome the limitations of the single linked list:

- **Doubly Linked List:** Each node contains *two* pointers—one pointing to the next node and one pointing to the previous node. This enables efficient bidirectional traversal and simplifies node deletion, at the cost of increased memory overhead per node.
- **Circular Linked List:** The pointer of the final node (the tail) does not point to null; instead, it loops back and points to the first node (the head), forming a continuous loop. This is heavily utilized in operating systems for round-robin scheduling.